

Contenuti

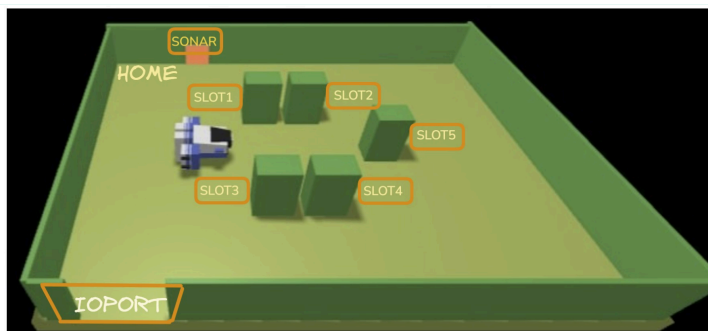
Sprint 0: analisi dei requisiti, descrizione e modellazione dei componenti, piano di test e goal per Sprint 1.

Requisiti

A *Maritime Cargo shipping company* (from now on, simply **company**) intends to automate the operations of load of **containers** in the ship's cargo hold (or simply **hold**). To this end, the company plans to employ a *Differential Drive Robot* (from now, called **cargorobot**). The **hold** is a rectangular, flat area with an Input/Output port (**IOPort**). The area provides 4 *slots* to store the containers and a slot named **slot5**.

In the picture below:

- The **slots1-4** depict the hold areas reserved to store one container each
- The **slot5** depicts an area, where the cargorobot must temporarily store a container, before to place it in one of the slots1-4. During temporary storage, a 'marker' device labels the container with an identification barcode and signals when this marking activity is completed
- The **IOPort** is a device with a **pushbutton** and a **display**. The pushbutton is pressed by the customer in order to to send a request to load a container on the cargo. The display is used to show the answer to the request and to show the current state of the hold
- The **sensor** associated to the IOPort is a device (a sonar) used to detect the presence of a container, when it measures a distance D , such that $D \leq D_{FREE}/2$, during a reasonable time (e.g. 3 secs)



The company asks us to build service named **cargoservice** that should work as follows.

The *cargoservice* is able to receive a **request to load** a container sent by some customer by using the *pushbutton* of the *IOPort*.

- It sends the answer **retrylater**, if the **IOPort** is currently occupied by a container or if the system is *Out of service*
- It rejects the request when the hold is already full, i.e. the **slots1-4** are already occupied.
- Otherwise, it considers the system as **engaged**, detects a free slot and returns as answer the name of such a reserved slot. While engaged, the system must blink a Led.

When the *request to load* is accepted, the customer must move the container in the *sensor* area within prefixed amount of time (e.g. 30 secs), otherwise the systems becomes **disengaged**. Then, the *cargoservice* uses the *cargorobot* to move the container from the

IOPort to the *slot5* (for marking the container) and then to the reserved slot.

The service must also show on the *display* on the *IOPort*:

- the current state of the *hold*
- the message '**Service working**', when it is all is going well
- the message '**Out of service**' if the *sonar sensor* measures a distance $D > = D_{FREE}$ for at least 3 secs (perhaps a failure of the sonar).

Premessa: utilizzo di QAK

L'analisi dei requisiti di questo sprint iniziale si fonda, oltre che sull'utilizzo di classi java, anche sul concetto di attore e sull'utilizzo del metamodello QAK.

Adottare il concetto di attore nella fase di analisi ci ha consentito di meglio rappresentare le entità attive del sistema, quelle capaci di interagire proattivamente con altre attraverso lo scambio di messaggi. Un approccio basato esclusivamente su POJO avrebbe introdotto un eccessivo abstraction gap tra le entità reali da modellare e gli strumenti di rappresentazione disponibili.

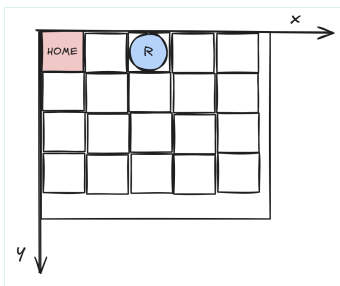
Tra i concetti offerti da QAK, i più rilevanti ai fini del progetto sono secondo noi le diverse tipologie di messaggi scambiabili tra gli attori del sistema, ciascuno con la propria sintassi e semantica.

Analisi dei Requisiti

Hold e slots

La **hold** è lo spazio di lavoro del cargorobot. Si tratta di uno spazio rettangolare di altezza **H** e base **B** (deducibili da figura nei requisiti) contenente dei punti di interesse per la logica di business, in particolare gli slots, anch'essi con una posizione definita (mostrata in figura da requisiti). Questo dai requisiti ci è sembrato un componente passivo, ampiamente legato alla logica dei dati della sua dimensione, degli slot e dei carichi. Per questo abbiamo pensato di utilizzare la classe java **Hold** per formalizzarne il modello (non necessariamente una scelta di progetto o implementazione avanzata!!!).

Per rappresentare lo spazio rettangolare dell'hold e l'esatto posizionamento degli slot al suo interno, abbiamo pensato di utilizzare un sistema di coordinate discrete, ottenute astruendo il robot che si muoverà al suo interno come un cerchio di raggio **r**, e quindi suddividendo l'area della hold in quadrati di lato $2r$. Questo si riflette nel modello come una matrice di interi `int[][] hold`, di dimensione $\text{width}(B/2r) * \text{height}(H/2r)$.



Decidiamo di modellare i punti di interesse slot come indici interi $i[1..n+1]$, che verranno inseriti staticamente all'interno della matrice, mentre gli spazi "vuoti" di hold verranno modellati come 0. Per rappresentare questo abbiamo deciso di riportare un possibile costruttore della nostra classe **Hold** che riceve, fra le altre cose, `int n` (numero di slots) e `slots_coordinates[n][2]` (coordinate dei singoli slots all'interno dell'**hold**).

Da requisiti, ogni slot può essere riservato o libero, perciò abbiamo pensato di modellare questo aspetto come un array di boolean di **n** elementi `int[] slots_reservations`, corrispondente agli stati degli slot (true = riservato, false = libero). Questo stato può essere sovrascritto tramite le funzioni pubbliche `void freeSlot(int slot_id)` e `void reserveSlot(int slot_id)`. Lo stato dei singoli slot può essere ispezionato tramite `boolean isSlotReserved(int slot_id)`.

```
public Class Hold{
    private int[][] hold;
    private int width;    // width*2r < B < width*(2r+1)
    private int height;   // height*2r < H < height*(2r+1)
    private int number_of_slots n;
    private boolean slots_reservations[];
```

```

public Hold(int B, int H, int r, int n, int[][] slots_coordinates){
    //inizializzazione dimensioni dell'hold da dati in input
    width = B/(2*r);
    height = H/(2*r);
    hold = new int[H][W];

    //inizializzazione dello spazio dell'hold a 0, quindi vuoto
    for(int i = 0; i < height; i++){
        for(int j = 0; j < width; j++){
            hold[i][j] = 0;
        }
    }

    this.n = n;
    slots_reservations = new boolean[n];

    //posizionamento statico degli slots e inizializzazione reservation a false, free
    for(int i = 0; i < n; i++){
        int slot_x = slots_coordinates[n][0];
        int slot_y = slots_coordinates[n][1];

        hold[slot_y][slot_x] = n+1;

        slots_reservations[n] = false;
    }
}

public void freeSlot(int slot_id){
    slots_reservations[slot_id - 1] = false;
}

public void reserveSlot(int slot_id){
    slots_reservations[slot_id - 1] = true;
}

public boolean isSlotReserved(int slot_id){
    return slots_reservations[slot_id - 1];
}
}

```

IOPort

La **IOPort** è il punto di ingresso/uscita dei container nella hold.

Logicamente, la IOPort è composta da:

- **pushbutton**: pulsante fisico. Premuto dal cliente, invia una **richiesta di carico** al cargoservice. (attivo)
- **display**: mostra risposta e stato della hold. (passivo)
- **sensor**: periodicamente rileva la distanza D nella **IOPort**, e quando questa è minore di $=DFREE/2$ per almeno 3 secondi è in grado di dedurre la presenza di un ostacolo. (attivo)

Natura software: già da requisiti è possibile capire come **IOPort** sia un componente complesso, composto da elementi proattivi come **pushbutton** e **sensor**, che devono essere in grado di inviare autonomamente una comunicazione, e passivi come **display**, il cui stato può solo essere modificato dall'esterno.

Per questo abbiamo pensato di modellare **display** come un POJO (classe **Display**)

pushbutton e **sensor** verranno invece modellati come attori QAK.

Display

display è un componente che deve essere in grado di:

- Mostrare la risposta alla richiesta di carico conseguente alla pressione di **pushbutton**: "Accepted" oppure "Retry Later"
- Mostrare lo "stato corrente" della **hold**
- Mostrare lo stato di funzionamento del **sonar sensor**: "Service working" oppure "Out of Service"

????Cosa intende il committente con "stato corrente" della hold??????????

Decidiamo dunque di modellarlo con l'astrazione di POJO (classe **Display**), dotato di:

- Metodo **updateHoldStatus** per l'aggiornamento dello stato della **hold**. Questo riceve in input lo stato aggiornato mediante Stringa di caratteri e va a stamparlo.
- Metodo **updateReply**, che riceve un valore booleano **reply** (true= Accepted, false= Retry Later) e va a stampare il messaggio corrispondente
- Metodo **updateSonarStatus**, che riceve un valore booleano **status** (true= Working, false= Out of Service) e va a stampare il messaggio corrispondente

```
public Class Display{
    public void updateHoldStatus(String status){
        .....
    }

    public void updateReply(boolean reply){
        String replyMessage = reply ? "Request Accepted" : "Retry Later";
        System.out.println(replyMessage);
    }

    public void updateSonarStatus(boolean status){
        String statusMessage = reply ? "System Working" : "Out of Service";
        System.out.println(statusMessage);
    }
}
```

Pushbutton

pushbutton è un componente che deve essere in grado di, un volta premuto, inviare una richiesta di carico a cargoservice

E' quindi un componente attivo, in grado di iniziare un'interazione con altri componenti del sistema. Decidiamo di modellarlo come un attore QAK in grado di inviare una Request (nei requisiti è esplicitamente detto che **IOPort** si aspetta una risposta per aggiornare **display**) **requestToLoad** una volta premuto.

(qua QAK)

Sensor

Il **sensor** è un componente **attivo** che rileva la presenza o l'assenza di un container presso la IOPort, misurando una distanza D.

Inoltre, data una distanza =DFREE, valgono le seguenti proprietà:

D < =DFREE/2	= container presente, IOPort occupata
=DFREE/2 <= D <= DFREE	= misurazione valida, IOPort libera
D > =DFREE per più di 3 sec	= sistema entra in Out-of-Service

Comunque decidiamo di modellare **sensor** come una classe java **Sensor** avente un main con un loop infinito in cui controlla la distanza con un metodo (stub) private int **checkDistance()** e che aggiorna due variabili boolean di stato **outOfService** e **occupied**.

```
public class Sensor(){
    private boolean outOfService;
    private boolean occupied;
    private int static DFREE;

    public static void main(String[] args){
        int occupiedCounter = 0;
        int outOfServiceCounter = 0;
        while(true){
            int D = checkDistance();

            System.out.println("Distance: "+D);
            time.wait(1);

            if(D >= DFREE/2 && D <= DFREE){
                counterOutOfService = 0;
                counterOccupied = 0;
                outOfService = false;
                occupied = false;
            } else if(D < DFREE){
                counterOccupied++;
                counterOutOfService = 0;
                outOfService = false;
            } else {
```

```

        counterOutOfService++;
        counterOccupied = 0;
        occupied = false;
    }

    if(counterOccupied >= 3){
        occupied = true;
    }

    if(counterOutOfService >= 3){
        outOfService = true;
    }

    }

}

private int checkDistance(){
    int measuredDistance;
    .....
    return measuredDistance;
}
}

```

Led

Il **led** è un dispositivo passivo di notifica visiva. I suoi comportamenti:

- **BLINK**: stato **ENGAGED**
- **OFF**: stati **DISENGAGED/OUT OF ORDER**

In questa fase di analisi dei requisiti modelliamo il led come una classe java, avente una variabile booleana blink che ne rappresenta lo stato (true = engaged, false = disengaged/out of order). Il led può essere acceso/spento tramite i metodi void blink() e off(). Il suo stato può essere verificato tramite il metodo boolean isBlinking().

```

public class Led(){
    private boolean blink = false;

    public void blink(){
        blink = true;
    }

    public void off(){
        blink = false;
    }

    public boolean isBlinking(){
        return blink;
    }
}

```

Il core business

Il **cargoservice** è il macro-componente che implementa il core-business e Orchestra tutti gli altri componenti, modellato quindi come **ATTORE**.

La logica di business principale è composta da:

- dove e quando spostare il cargorobot
- accettare o rifiutare una richiesta di carico
- quale slot libero associare alla richiesta che si sta servendo
- mandare out of order il sistema quando si accorge di un malfunzionamento del sensor

Architettura Logica del Sistema

Scegliamo di utilizzare il domain-specific language QAK per modellare i componenti dell'architettura e le loro interazioni in quanto linguaggio volto non solo alla programmazione ma anche alla descrizione e analisi dei sistemi.

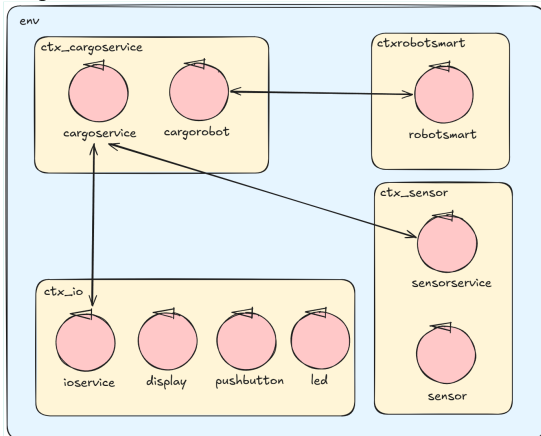
In questa fase decidiamo di fare qualche scelta di progetto. Innanzitutto,

L'architettura logica è descritta nell'immagine. Abbiamo deciso di creare l'attore **cargorobot** per astrarre il dominio di robotsmart: cargoservice ha lo scopo di comunicare le posizioni astratte come slot1-5, home, ioport senza conoscere il sistema di coordinate. Inoltre, se si vuole utilizzare un robot reale basterà modificare solo le chiamate di cargorobot senza cambiare la

logica di business di cargoservice.

Il **ioservice** è un attore fondamentale per comunicare con le periferiche hardware (pushbutton, display, led).

Il **sensorservice** (contesto dedicato **ctx_sensor**, insieme al **sensor**) si occupa di gestire il sensor e inviare al cargoservice solo gli eventi generati dal sensor; in questo modo cargoservice riceve solo le informazioni utili (ad esempio quando l'utente deposita il container), al posto di inviare i dati grezzi della distanza D. In questo modo abbiamo una divisione netta dei ruoli e dei domini.



I messaggi principali scambiati tra/nei contesti del sistema sono:

cargoservice (ctx_cargoservice) ↔ ioservice (ctx_io)

- Richieste di carico: il pushbutton notifica ioservice, che inoltra a cargoservice la richiesta di loading:

```
Request  load_request  : load_request(X)
Reply    load_accepted : load_accepted(SLOT)   for load_request //1 | 2 | 3 | 4
Reply    load_rejected : load_rejected(REASON) for load_request //full | out_of_service
Reply    retrylater    : retrylater(X)         for load_request
```

- Comandi da cargoservice verso ioservice:

```
Dispatch setDisplay : setDisplay(MSG) //testo sul display
Dispatch setLed     : setLed(STATE)   //off | blink
```

cargoservice (ctx_cargoservice) ↔ sensorservice (ctx_sensor)

- Verifica **sincrona** (request-reply) della presenza di un container all'IOPort. Necessaria perché **containerIn** è un dispatch **fire and forget**: se il container è stato depositato **prima** della load_request, il dispatch è già stato emesso (e perso) e cargoservice non può sapere se il container c'è:

```
Request  check_container : check_container(X)
Reply    container_present : container_present(X) for check_container
Reply    container_absent  : container_absent(X)  for check_container
```

- Notifiche **asincrone** (fire and forget) da sensorservice. Se la verifica sincrona risponde **container_absent**, cargoservice attende il dispatch **containerIn** entro 30s:

```
Dispatch containerIn : containerIn(X) //container rilevato all'IOPort
Dispatch containerOut : containerOut(X) //container rimosso
Dispatch sensorError  : sensorError(X) //sensor guasto: outOfService
Dispatch sensorReady  : sensorReady(X) //sensor ok: recovery da outOfService
```

sensor → sensorservice (interno a ctx_sensor)

- Dati grezzi, mai inoltrati a cargoservice:

```
Dispatch sensordata : sensordata(D) //distanza misurata, periodica
```

cargoservice ↔ cargorobot (interno a ctx_cargoservice)

- Movimentazione con destinazione logica (nessuna coordinata):

```
Request  do_move : do_move(DEST) //home|slot1..5|ioport
Reply    move_done : move_done(ARG) for do_move
Reply    move_failed : move_failed(CAUSE) for do_move
```

cargorobot (ctx_cargoservice) ↔ **robotsmart** (ctxrobotsmart)

- Pianificazione ed esecuzione del percorso:

```
Request  buildPlan    : buildPlan(PX,PY,TX,TY)
Reply    buildPlanDone : buildPlanDone(PLAN)  for buildPlan
Request  doplan       : doplan(PPLAN, STEPTIME)
Reply    doplandone    : doplandone(ARG)      for doplan
Reply    doplanfailed  : doplanfailed(PLANTODO) for doplan
```

Componenti da Sviluppare

Da sviluppare

- **cargoservice**: Sprint 1 (core business, gestione stati)
- **cargorobot**: Sprint 1 (astrazione movimentazione robotsmart)
- **hold** : Sprint 1 (gestione mappa e slot)
- **sensorservice** : Sprint 2 (attività del sensore a cargoservice)
- **ioservice** : Sprint 3 (gestione io hw)
- **pushbutton** : Sprint 3 (input virtuale/hw)
- **display** : Sprint 3 (output verso utente / webGUI)
- **led** : Sprint 3 (notifica visiva virtuale/hw)

Forniti dal committente

- **robotsmart26** (sistema QAK)
- **WEnv** (VirtualRobot26, Docker)
- **sensor ???**

Test Plans

I test sono scritti in Java (JUnit). Sono dei test preliminari, verranno implementati nello Sprint1 e potrebbero cambiare sintatticamente.

Prima di ogni test il sistema è avviato e **cargoservice** è raggiungibile.

sensorservice è simulato: i dispatch **containerIn**, **sensorError**, **sensorReady** vengono iniettati direttamente dal tester, mentre uno **stub** di sensorservice risponde alle **check_container**.

T01: Richiesta accettata - ENGAGED

Precondizione: sistema in DISENGAGED, almeno uno slot1-4 libero.

Atteso: cargoservice risponde **load_accepted(slotN)** e passa a ENGAGED.

```
@Test
public void testT01() throws Exception {
    String req = CommUtils.buildRequest("tester", "load_request", "load_request(client1)", "cargoservice").toString();
    String resp = conn.request(req);

    assertTrue("T01: load_accepted con slot assegnato", resp.contains("load_accepted"));
}
```

T02: Container depositato - normale operazione - DISENGAGED (normale flow di operazione)

Precondizione: sistema in ENGAGED (T01 eseguito), IOPort libera al momento della richiesta.

Atteso: dopo **containerIn** da sensorservice, cargorobot esegue il ciclo IOPORT - SLOT5 - slotN, il sistema torna in DISENGAGED con lo slot occupato.

```
@Test
public void testT02() throws Exception {

    //porta il sistema in ENGAGED
```

```

String req = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
conn.request(req);

//simula sensorservice: container rilevato all'IOPort (dispatch asincrono)
conn.forward(CommUtils.buildDispatch("tester", "containerIn", "containerIn(x)", "cargoservice").toString());

CommUtils.delay(8000); //attendo finisca la normale operazione

//una nuova richiesta deve essere accettata (sistema tornato in DISENGAGED)
String req2 = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
String resp2 = conn.request(req2);

assertTrue("T02: sistema tornato in DISENGAGED dopo carico", resp2.contains("load_accepted"));
}

```

T03: Timeout in ENGAGED - DISENGAGED - slot liberato

Precondizione: sistema in ENGAGED, un solo slot libero.

Atteso: allo scadere dei 30s senza **containerIn**, il sistema torna in DISENGAGED con lo slot liberato (prima riservato) e risponde **retrylater** alla richiesta originale.

```

@Test
public void testT03() throws Exception {
    String req = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    String resp = conn.request(req);
    assertTrue("T03 setup: richiesta accettata", resp.contains("load_accepted"));

    //non piazza un container, attendo scadenza timer (5 secondi per testare)
    CommUtils.delay(6000);

    //lo slot deve essere stato liberato
    String req2 = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    String resp2 = conn.request(req2);

    assertTrue("T03: slot liberato dopo timeout", resp2.contains("load_accepted"));
}

```

T04: Hold piena - load_rejected

Precondizione: tutti gli slot1-4 occupati.

Atteso: cargoservice risponde **load_rejected**.

```

@Test
public void testT04() throws Exception {

    //invio richiesta di carico
    String req = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    String resp = conn.request(req);

    assertTrue("T04: hold piena - load_rejected", resp.contains("load_rejected"));
}

```

T05: Sensor guasto - Out of Service - retrylater

Precondizione: sistema in DISENGAGED.

Atteso: dopo **sensorError** da sensorservice il sistema entra in OUT_OF_SERVICE, le richieste successive ricevono **retrylater(out_of_service)**.

```

@Test
public void testT05() throws Exception {
    //simulo sensor guasto
    conn.forward(CommUtils.buildDispatch("tester", "sensorError", "sensorError(x)", "cargoservice").toString());
    CommUtils.delay(300);

    //invio richiesta di carico
    String req = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    String resp = conn.request(req);

    assertTrue("T05: out_of_service - retrylater", resp.contains("retrylater"));
}

```


T06: Recovery da Out-of-Service

Precondizione: sistema in OUT_OF_SERVICE (T05 eseguito).

Atteso: dopo **sensorReady** da sensorservice il sistema torna in DISENGAGED e accetta nuove richieste.

```
@Test
public void testT06() throws Exception {
    //porto il sistema in OUT_OF_SERVICE
    conn.forward(CommUtils.buildDispatch("tester", "sensorError", "sensorError(x)", "cargoservice").toString());
    CommUtils.delay(300);

    //simulo fine guasto
    conn.forward(CommUtils.buildDispatch("tester", "sensorReady", "sensorReady(x)", "cargoservice").toString());
    CommUtils.delay(300);

    //invio richiesta di carico
    String req = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    String resp = conn.request(req);

    assertTrue("T06: recovery - sistema torna in DISENGAGED", resp.contains("load_accepted"));
}
```

T07: Container già presente all'IOPort - carico immediato

Precondizione: container depositato **prima** della load_request (il dispatch **containerIn** è fire and forget ed è già andato perso).

Atteso: alla load_request, cargoservice interroga sensorservice con **check_container**, lo stub risponde **container_present** e il ciclo di movimentazione parte subito, senza attendere i 30s.

```
@Test
public void testT07_ContainerAlreadyAtIoport() throws Exception {
    //setto lo stub del sensorservice in stato "container presente"
    sensorStub.setContainerStatus(true);

    //invio richiesta di carico
    String req = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    String resp = conn.request(req);
    assertTrue("T07 setup: richiesta accettata", resp.contains("load_accepted"));

    CommUtils.delay(8000); //attendo finisca l'operazione

    //sistema tornato in DISENGAGED: nuova richiesta accettata
    sensorStub.setContainerStatus(false);
    String req2 = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    String resp2 = conn.request(req2);

    assertTrue("T07: carico avviato con container già presente", resp2.contains("load_accepted"));
}
```

Goal Sprint 1

Lo **Sprint 1** ha come obiettivo la realizzazione del core business, per ora senza dipendere da IOPort e dal sensore.

Sottoinsieme di Requisiti (goal Sprint 1)

- Implementare **cargoservice**
- Implementare **cargorobot**
- Gestire la riserva degli slot e il ciclo di movimentazione
- Gestire la verifica sincrona **check_container** verso lo stub di sensorservice
- Gestire il timeout di 30 secondi per il deposito (attesa asincrona di **containerIn** da sensorservice)
- Sensor simulato (stub di sensorservice)
- Hold e slot (POJO)

Test da superare in Sprint 1

- **T01**: Richiesta accettata
- **T02**: Container depositato in tempo
- **T03**: Timeout / DISENGAGED
- **T04**: Hold piena
- **T05**: Sensor guasto / OUT_OF_SERVICE
- **T06**: Recovery da OUT_OF_SERVICE
- **T07**: Container già presente all'IOPort (check sincrono)

Componenti prodotti in Sprint 1

- **cargoservice**
- **cargorobot**
- **hold**

Piano di lavoro

- Prevediamo al massimo 20 ore/uomo, su una settimana di lavoro.
- Francesco si occuperà principalmente dell'implementazione QAK.
- Andrea si occuperà principalmente dei Test e stesura del report.
- Entrambi ci aggiorniamo giornalmente sul lavoro svolto, studiamo e valutiamo i rispettivi lavori.



By Andrea Gazzola

and Francesco Roffia



[GIT repo:](#)

https://github.com/FRoffia/ISS_26_Cargoservice